

Содержание

1. Глоссарий	4
1.1. API	4
1.2. BI	4
1.3. CDN	4
1.4. Компонент	4
1.5. CRM	4
1.6. СУБД	4
1.7. Эмбединг	4
1.8. Индекс	4
1.9. Ввод-вывод	4
1.10. IoT	4
1.11. Слой	4
1.12. Метаданные	5
1.13. OLAP	5
1.14. OLTP	5
1.15. Сессия	5
1.16. SLA	5
1.17. Support	5
1.18. Транзакция	5
2. Архитектура информационных систем	6
2.1. Модуль 1: Архитектурные основы	6
2.1.1. Лекция 1: Введение. Принципы построения приложения	6
2.1.1.1. 1. Компонент как класс (C++)	6
2.1.1.2. Зависимости компонентов и слои	7
2.1.1.3. Продуктивность слоя	8
2.1.1.4. Оптимизация системы требует выбора критерия оптимизации	9
2.1.1.5. Принципы SOLID	9
2.1.1.6. Dependency Injection	9
2.1.1.7. Clean Architecture	9
2.1.2. Лекция 2: Альтернативные архитектурные паттерны	10
2.1.2.1. Архитектура на основе фреймворка (Rails, Django)	10
2.1.2.2. Модель акторов (Erlang/Elixir/Akka)	10
2.1.2.3. Проектирование, ориентированное на данные (Data-oriented design)	10
2.1.3. Лекция 3: Протоколы взаимодействия: REST / gRPC	10
2.1.3.1. REST: стандартный веб-протокол	10
2.1.3.2. gRPC: бинарный протокол, высокая производительность	10
2.1.4. Лекция 4: Протоколы взаимодействия: GraphQL / WebSocket	11
2.1.4.1. GraphQL: клиент запрашивает только нужные данные	11
2.1.4.2. WebSocket: двусторонний канал	11
2.2. Модуль 2: Фундаментальные структуры хранилищ данных	12
2.2.1. Лекция 5: Классификация нагрузки	12
2.2.1.1. Типы операций	12
2.2.1.2. Метрики нагрузки	12
2.2.1.3. Характеристики нагрузки	12
2.2.1.4. Оценка QPS	13
2.2.1.5. Оценка объема хранения	14
2.2.1.6. Иерархия памяти	14
2.2.1.7. Фундаментальные компромиссы	14
2.2.2. Лекция 6: LSM Дерево	15
2.2.2.1. Общая идея	15
2.2.2.2. Отсортированная таблица строк (Sorted String Table, SSTable)	15

2.2.2.3. Memtable	15
2.2.2.4. Bloom Filter	16
2.2.2.5. Полная структура LSM-дерева	16
2.2.2.6. Практическое применение	17
2.2.3. Лекция 7: B-дерево: индексирование на диске	18
2.2.3.1. B-фактор и структура узла	18
2.2.3.2. Практическое применение	19
2.2.4. Лекция 8: ACID, уровни изоляции, аналитические и транзакционные хранилища	21
2.2.4.1. Транзакции	21
2.2.4.2. ACID (Atomicity, Consistency, Isolation, Durability)	21
2.2.4.3. Транзакционные и аналитические хранилища	21
2.2.4.4. Сжатие колонок	22
2.2.5. Лекция 9: Хранилища в оперативной памяти	23
2.2.5.1. Аналитика: DuckDB и векторизация	23
2.2.5.2. Кэширование: Redis	23
2.3. Модуль 3: Методы поиска похожих/близких элементов	24
2.3.1. Лекция 10: Специализированные индексы для поиска	24
2.3.1.1. Inverted Index (перевёрнутые индексы)	24
2.3.1.2. Trie (префиксные деревья)	24
2.3.1.3. Bitmap indexes	25
2.3.2. Лекция 11: Векторные индексы для поиска подобия	26
2.3.2.1. Основы векторных представлений	26
2.3.2.2. HNSW (Hierarchical Navigable Small World)	27
2.3.2.3. LSH (Locality-Sensitive Hashing)	28
2.3.2.4. Примеры: Weaviate, Pinecone, Qdrant	28
2.3.3. Лекция 12: Геопространственные структуры	28
2.3.3.1. R-Tree	28
2.3.3.2. QuadTree и KD-Tree	29
2.4. Модуль 4: Операционные компоненты	29
2.4.1. Лекция 13: Прокси и Rate Limiting	29
2.4.1.1. Forward Proxy / Reverse Proxy	29
2.4.1.2. Распределение нагрузки	29
2.4.1.3. Ограничение частоты запросов	30
2.4.2. Лекция 14: Безопасность и управление доступом	30
2.4.2.1. Аутентификация и авторизация	30
2.4.2.2. RBAC и ABAC	30
2.4.2.3. Single Sign-On	30
2.4.3. Лекция 15: Логирование, мониторинг и планирование задач	30
2.4.3.1. Сбор логов	30
2.4.3.2. Сбор метрик (Prometheus)	30
2.4.3.3. Мониторинг и диагностика (Grafana)	30
2.4.3.4. Планировщики (Celery, Airflow, Dagster)	30
2.5. Модуль 5: System Design	30
2.5.1. Лекция 16: Разбор дизайна конкретной системы	30
2.5.1.1. Применение всех концепций на практике	30
3. Распределённые системы обработки данных	31
3.1. Модуль 1: Компоненты распределённых систем	31
3.1.1. Лекция 1: Основы распределённых систем	31
3.1.1.1. CAP-теорема	31
3.1.1.2. Механизмы репликации данных	31
3.1.1.3. ZooKeeper: распределённое решение проблем согласованности и отказоустойчивости ..	
3.1.2. Лекция 2: Гарантии консистентности и координация распределённых транзакций	31

3.1.2.1. Алгоритмы достижения консенсуса	31
3.1.2.2. Модель конечной консистентности (Eventual consistency) и требование идемпотентности операций	31
3.1.2.3. Протокол двухфазного коммита (Two-Phase Commit, 2PC)	31
3.2. Модуль 2: Распределенные сервисы	31
3.2.1. Лекция 3: Асинхронное взаимодействие сервисов	31
3.2.1.1. Event Bus и Event-Driven Architecture	31
3.2.1.2. Publish-Subscribe vs Message Queue	31
3.2.1.3. Saga Pattern	31
3.2.2. Лекция 4: Kafka	31
3.2.2.1. Внутреннее устройство	31
3.2.3. Лекция 5: Kubernetes	31
3.2.3.1. Роль и место в современных архитектурах	31
3.2.3.2. Области использования	31
3.2.3.3. Внутреннее устройство	31
3.2.4. Лекция 6: ClickHouse	31
3.2.4.1. Архитектура системы хранения и обработки данных	31
3.2.4.2. Партиционирование	31
3.2.4.3. Таблицы серии MergeTree как основной механизм персистентности	32
3.2.5. Лекция 7: Apache Spark	32
3.2.5.1. Роль и место в современных архитектурах	32
3.2.5.2. Области использования	32
3.2.5.3. Внутреннее устройство	32
3.2.5.4. API	32
3.2.6. Лекция 8: Erlang	32
3.2.6.1. Роль и место в современных архитектурах	32
3.2.6.2. Erlang VM (BEAM) и процессы	32
3.2.6.3. Модель акторов	32
3.2.6.4. Fault tolerance и “Let it crash” философия	32
3.2.6.5. OTP фреймворк (Supervisor, GenServer)	32
3.2.6.6. Синхронизация и распределённые вычисления	32
3.2.6.7. Elixir	32
3.3. Модуль 3: Распределенные БД	32
3.3.1. Лекция 9: Большие данные	32
3.3.1.1. Свойства	32
3.3.1.2. Архитектуры	32
3.3.2. Лекция 10: Full Text Search в распределённых системах	32
3.3.2.1. Elasticsearch и Solr	32
3.3.2.2. Распределённый поиск	32
3.3.2.3. Масштабирование	32
3.3.3. Лекция 11: Документные хранилища	33
3.3.3.1. MongoDB: sharding strategies, write concerns	33
3.3.3.2. Object storage: MinIO	33
3.3.4. Лекция 12: Распределенные пространственные и графовые БД	33
3.3.4.1. Графовые БД	33
3.3.4.2. Пространственные БД	33
3.3.4.3. Шардирование графов и пространственных данных	33

1. Глоссарий

1.1. API

Application Programming Interface — программная абстракция, определяющая методы и протоколы взаимодействия между программными компонентами. Обеспечивает унифицированный доступ к функциональности системы посредством стандартизированных вызовов, скрывая детали внутренней реализации.

1.2. BI

Business Intelligence — технологии и практики сбора, интеграции, анализа и представления бизнес-информации. Включает инструменты для создания отчётов, дашбордов, визуализации данных и поддержки принятия управленческих решений на основе данных.

1.3. CDN

Content Delivery Network — географически распределённая сеть серверов для кэширования и доставки статического контента пользователям с минимальной задержкой.

1.4. Компонент

Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

1.5. CRM

Customer Relationship Management — система управления взаимоотношениями с клиентами. Программное обеспечение для автоматизации стратегий взаимодействия с клиентами, включая продажи, маркетинг, техподдержку и аналитику клиентских данных.

1.6. СУБД

Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

1.7. Эмбединг

Векторное представление объекта (текста, изображения, пользователя, товара), полученное моделью машинного обучения для измерения семантической близости через расстояния в векторном пространстве.

1.8. Индекс

Вспомогательная структура данных в системах управления базами данных, предназначенная для оптимизации производительности операций поиска и извлечения информации. Организует упорядоченное представление данных для существенного сокращения времени доступа к записям.

1.9. Ввод-вывод

Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

1.10. IoT

Internet of Things — сетевое взаимодействие физических устройств с датчиками и контроллерами. Характеризуется высокой частотой передачи небольших порций данных (телеметрия, метрики).

1.11. Слой

Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

1.12. Метаданные

Структурированная информация, описывающая свойства других данных: время создания, размер, тип, владелец, права доступа. Используется для индексирования и маршрутизации без чтения основного содержимого.

1.13. OLAP

Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объёмам данных, использует колоночное хранение для оптимизации производительности аналитики.

1.14. OLTP

Online Transaction Processing — класс систем обработки транзакций в реальном времени. Характеризуется большим количеством коротких транзакций (чтение, вставка, обновление, удаление), строковым хранением данных и требованиями к консистентности.

1.15. Сессия

Логический контекст взаимодействия клиента с системой между множественными запросами. Хранит состояние пользователя: аутентификацию, корзину, настройки. Реализуется через cookies + серверное хранилище (Redis) или JWT-токены.

1.16. SLA

Service Level Agreement — соглашение об уровне обслуживания с количественными метриками: uptime (99.9% = 8.76 ч простоя/год), задержка (p99 < 200 мс).

1.17. Support

Служба поддержки и обработка обращений пользователей: тикеты, инциденты, запросы на возврат и консультации.

1.18. Транзакция

Последовательность операций чтения и записи в базе данных, рассматриваемая как единая логическая единица работы. Гарантирует атомарность выполнения: либо все операции завершаются успешно и фиксируются (COMMIT), либо при ошибке все изменения откатываются (ROLLBACK).

2. Архитектура информационных систем

2.1. Модуль 1: Архитектурные основы

2.1.1. Лекция 1: Введение. Принципы построения приложения

Компонент — Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

Рассмотрим несколько примеров:

2.1.1.1. 1. Компонент как класс (C++)

```
#include <iostream>
```

```
class NumberPrinter {
public:
    void print(int x) {
        std::cout << x << std::endl;
    }

    void printRange(int from, int to) {
        for (int i = from; i <= to; ++i) {
            std::cout << i << std::endl;
        }
    }
};
```

Единица вычисления: форматирование и вывод чисел

Интерфейс: сигнатуры публичных методов

Зависимость: консоль ввода-вывода

Форма реализации: класс (ООП-модель)

- Компонент как функция (Elixir)

```
def send_message(host, port, message) do
    {:ok, socket} = :gen_tcp.connect(host, port, [:binary, active: false])
    :ok = :gen_tcp.send(socket, message)
    :gen_tcp.close(socket)
end
```

Единица вычисления: передача сообщения

Интерфейс: сигнатура функции

Зависимость: сеть (TCP stack)

Форма реализации: функция (функциональная модель)

- Компонент как функция (Go)

```
func LoadUserNames(db *sql.DB) ([]string, error) {
    rows, err := db.Query("SELECT name FROM users")
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var result []string
    for rows.Next() {
        var name string
        if err := rows.Scan(&name); err != nil {
            return nil, err
        }
    }
}
```

```

    result = append(result, name)
}
return result, nil
}

```

Единица вычисления: извлечение данных

Интерфейс: сигнатура функции

Зависимость: БД (SQL, соединение)

Форма реализации: функция (процедурная / Go-модель)

2.1.1.2. Зависимости компонентов и слои

Слой — Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

Рассмотрим пример слоя персистентности, который сохраняет различные модели в CSV:

```

// domain/models.go - слой моделей данных
package domain

type User    struct{ ID int; Name, Email string }
type Item    struct{ ID int; Title string; Price float64 }
type Storage struct{ ID int; Location string; Capacity int }

// persistence/csv_persistence.go - слой персистентности
package persistence

import (
    "fmt"
    "os"
    "strings"

    "example/domain" // Явная зависимость от слоя моделей
)

func SaveUsersToCSV(users []domain.User, filename string) error {
    lines := make([]string, 0, len(users))
    for _, u := range users {
        lines = append(lines, fmt.Sprintf("%d,%s,%s", u.ID, u.Name, u.Email))
    }
    return write(filename, lines)
}

func SaveItemsToCSV(items []domain.Item, filename string) error {
    lines := make([]string, 0, len(items))
    for _, i := range items {
        lines = append(lines, fmt.Sprintf("%d,%s,%v", i.ID, i.Title, i.Price))
    }
    return write(filename, lines)
}

func SaveStoragesToCSV(storages []domain.Storage, filename string) error {
    lines := make([]string, 0, len(storages))
    for _, s := range storages {
        lines = append(lines, fmt.Sprintf("%d,%s,%d", s.ID, s.Location, s.Capacity))
    }
    return write(filename, lines)
}

func write(filename string, lines []string) error {

```

```

return os.WriteFile(filename, []byte(strings.Join(lines, "\n")), 0644)
}

```

Анализ слоёв:

Слой	Компонент	Контракт (интерфейс)	Зависимости
domain	User	struct{ ID int; Name, Email string }	—
	Item	struct{ ID int; Title string; Price float64 }	—
	Storage	struct{ ID int; Location string; Capacity int }	—
persistence	SaveUsersToCSV	([]domain.User, string) error	domain.User, ФС
	SaveItemsToCSV	([]domain.Item, string) error	domain.Item, ФС
	SaveStoragesToCSV	([]domain.Storage, string) error	domain.Storage, ФС
	write	(string, []string) error	ФС

Заметим:

- **Слой моделей** (domain): три компонента-структуры без зависимостей
- **Слой персистентности** (persistence): четыре компонента-функции, три из которых явно зависят от типов слоя моделей
- Слой персистентности **зависит** от слоя моделей, но не наоборот

2.1.1.3. Продуктивность слоя

Прокси-компонент — компонент, не реализующий собственной вычислительной ответственности, а лишь делегирующий вызовы компонентам других слоёв.

Продуктивность слоя — количественная мера архитектурной самостоятельности слоя, определяемая долей компонентов, реализующих собственную вычислительную ответственность.

Формальное задание меры

Пусть:

- N — общее количество компонентов в слое
- P — количество прокси-компонентов в слое

Тогда коэффициент проксирования слоя определяется как:

$$K_{\text{проxy}} = \frac{P}{N}$$

а продуктивность слоя — как дополняющая величина:

$$K_{\text{prod}} = 1 - K_{\text{проxy}} = 1 - \frac{P}{N}$$

В стандартной литературе (например, Clean Architecture) слой считается продуктивным, если $K_{\text{prod}} \geq 0.8$, то есть не менее 80% компонентов реализуют собственную вычислительную ответственность.

Пример непродуктивного pass-through слоя:

Рассмотрим излишний слой сервисов, большинство компонентов которого только делегируют вызовы к слою персистентности:

```

// service/user_service.go - непродуктивный слой сервисов
package service

import (
    "example/domain"

```



```

    "example/persistence"
    "strings"
)

func GetUser(id int) (*domain.User, error) {
    return persistence.LoadUser(id)
}

func SaveUser(user *domain.User) error {
    return persistence.SaveUser(user)
}

func DeleteUser(id int) error {
    return persistence.DeleteUser(id)
}

func ListUsers() ([]*domain.User, error) {
    return persistence.ListUsers()
}

func SearchUsersByName(query string) ([]*domain.User, error) {
    users, err := persistence.ListUsers()
    if err != nil {
        return nil, err
    }

    var result []*domain.User
    for _, u := range users {
        if strings.Contains(strings.ToLower(u.Name), strings.ToLower(query)) {
            result = append(result, u)
        }
    }
    return result, nil
}

```

Анализ слоя сервисов:

Компонент	Контракт	Собственная ответственность
GetUser	(int) (*User, error)	Нет — прямая делегация
SaveUser	(*User) error	Нет — прямая делегация
DeleteUser	(int) error	Нет — прямая делегация
ListUsers	() ([]*User, error)	Нет — прямая делегация
SearchUsersByName	(string) ([]*User, error)	Да — фильтрация и поиск

Продуктивность слоя: $K_{\text{prod}} = 1 - \frac{4}{5} = 0.2$ (20% — непродуктивный слой)

Такой слой не добавляет архитектурной ценности и лишь усложняет кодовую базу, увеличивая количество уровней косвенности без предоставления дополнительной функциональности.

2.1.1.4. Оптимизация системы требует выбора критерия оптимизации

2.1.1.5. Принципы SOLID

2.1.1.6. Dependency Injection

2.1.1.7. Clean Architecture

2.1.2. Лекция 2: Альтернативные архитектурные паттерны

2.1.2.1. Архитектура на основе фреймворка (Rails, Django)

Фреймворк предполагает стандартную структуру папок, именование файлов и шаблоны проектирования (зачастую, MVC), что уменьшает количество принимаемых решений и ускоряет разработку.

- “Мнения” фреймворка: Rails/Django имеют предустановленные “мнения” по работе с БД (ORM), маршрутизацией, шаблонизацией и аутентификацией.

2.1.2.2. Модель акторов (Erlang/Elixir/Akka)

- Революционная технология, созданная в Ericsson еще в 1986 году для телекоммуникационных систем с требованием “пяти девяток” доступности.
- Она на десятилетия опередила современные подходы: предоставляет
 - встроенную распределенность (кластеры из коробки),
 - философия “Let it Crash!” с автоматическим самовосстановлением
 - горячее обновление кода без остановки системы
 - невероятную параллельность (миллионы изолированных процессов)
- WhatsApp: Всего 50 инженеров обрабатывали 2 миллиарда сообщений в день на одном сервере!
- Discord

2.1.2.3. Проектирование, ориентированное на данные (Data-oriented design)

Подход, фокусирующийся на эффективной организации данных в памяти для оптимального использования кэша процессора. Используется где производительность упирается в скорость доступа к памяти.

- игровые движки
- обработки физики
- AI
- симуляции

2.1.3. Лекция 3: Протоколы взаимодействия: REST / gRPC

API¹

2.1.3.1. REST: стандартный веб-протокол

- HTTP
 - методы (GET, POST, PUT, DELETE)
 - коды (Forbidden, Not Found)
- JSON
- URI Endpoints (например, /api/users/123)
- Swagger
- OpenAPI
 - Генерация спецификации из кода
 - Генерация кода из спецификации

2.1.3.2. gRPC: бинарный протокол, высокая производительность

- Историческая справка
 - Вырос из внутренней технологии Google под названием Stubby
 - Использовался для связи между тысячами микросервисов внутри дата-центров
 - В 2015 году открыли исходный код
- IDL (Описание формата общения в .proto-файлах)
- Генерация кода
- Мультиплексирование потоков (Stream Multiplexing)
- Двухнаправленная потоковая передача
- gRPC-Web прокси

¹Application Programming Interface — программная абстракция, определяющая методы и протоколы взаимодействия между программными компонентами. Обеспечивает унифицированный доступ к функциональности системы посредством стандартизированных вызовов, скрывая детали внутренней реализации.

2.1.4. Лекция 4: Протоколы взаимодействия: GraphQL / WebSocket

2.1.4.1. GraphQL: клиент запрашивает только нужные данные

Клиент формирует запрос с деревом полей, которые он хочет получить

- Проблема необходимости написания десятков различных запросов для получения одной сущности в различных экранах
- Подписка на реальные обновления данных
- Единственная конечная точка (Endpoint)

2.1.4.2. WebSocket: двусторонний канал

- Проблемы Long Polling
- Постоянное соединение, обмен сообщениями в реальном времени в обе стороны.
- Применение
 - Чат-приложения и онлайн-игры.
 - Биржевые тикеры и спортивные результаты.
 - Совместное редактирование документов.
 - Уведомления в реальном времени.

2.2. Модуль 2: Фундаментальные структуры хранилищ данных

Мы познакомились с архитектурными паттернами построения приложений и протоколами обмена данными между компонентами. Теперь возникает следующий вопрос: где и как хранить данные?

Задача выбора хранилища нетривиальна. На рынке сотни решений: реляционные базы, документные хранилища, key-value stores, колоночные базы, графовые базы, временные ряды. Каждое решение оптимизировано под определённый паттерн нагрузки, и неправильный выбор приводит к проблемам производительности, которые сложно исправить без миграции.

Чтобы осознанно выбирать между PostgreSQL и ClickHouse, между Redis и MongoDB, нужно понимать внутреннее устройство этих систем. В этом модуле мы разберём фундаментальные структуры данных, лежащие в основе современных хранилищ.

2.2.1. Лекция 5: Классификация нагрузки

При проектировании реальных систем первый шаг — оценка нагрузки. Без конкретных цифр невозможно принять обоснованное решение о выборе хранилища.

2.2.1.1. Типы операций

Зачастую взаимодействие с хранилищем сводится к ограниченному набору операций:

Тип	Операция	Пример
Чтение	Point lookup	SELECT * FROM users WHERE id = 123
	Multi-get	SELECT * FROM users WHERE id IN (1, 2, 3)
	Range scan	SELECT * FROM orders WHERE date BETWEEN '...' AND '...'
	Prefix scan	SELECT * FROM logs WHERE key LIKE 'user:123:%'
	Full scan	SELECT AVG(price) FROM products
Запись	Insert	вставка новой записи (ошибка при дубликате)
	Update	изменение существующей записи
	Upsert	INSERT ... ON CONFLICT DO UPDATE
	Delete	удаление записи
	Batch write	запись нескольких записей
Комб.	Read-modify-write	UPDATE accounts SET balance = balance + 100
	Increment	INCR views:page:123 (атомарный счётчик)

2.2.1.2. Метрики нагрузки

Метрика	Расшифровка	Описание
DAU	Daily Active Users	уникальные пользователи за сутки
MAU	Monthly Active Users	уникальные пользователи за месяц
RPS	Requests Per Second	пользовательских запросов (например HTTP) в секунду
QPS	Queries Per Second	запросов к БД в секунду
TPS	Transactions Per Second	транзакций БД в секунду
IOPS	I/O Operations Per Second	операций ввода-вывода в секунду

2.2.1.3. Характеристики нагрузки

Read/Write ratio:

- **90/10** — преобладание чтения: кэширование, CDN², статический контент
- **50/50** — сбалансированная: социальные сети, e-commerce
- **10/90** — преобладание записи: логирование, IoT³, метрики

²Content Delivery Network — географически распределённая сеть серверов для кэширования и доставки статического контента пользователям с минимальной задержкой.

Размер записи:

- < 1 KB — Метаданные⁴, Сессия⁵, счётчики
- 1-100 KB — JSON-документы, профили пользователей
- 100 KB - 1 MB — статьи, посты с изображениями
- > 1 MB — медиафайлы, бинарные данные

Паттерн доступа:

- **random** — point lookup по ключу, случайные запросы
- **sequential** — range scan, time-series, логи

Требования к латентности:

- **p50 < 50 мс** — медиана, типичный пользовательский опыт
- **p99 < 200 мс** — 1% худших запросов, важно для SLA⁶
- **p999 < 1 с** — 0.1% худших, критично для финтеха

Почему p99 важнее среднего: если p99 = 500 мс при 10000 QPS, то 100 запросов в секунду будут медленными — это 8.6 млн медленных запросов в сутки.

2.2.1.4. Оценка QPS

Пример расчёта — социальная сеть:

- 100 млн DAU (Daily Active Users)
- 300 млн MAU (Monthly Active Users)
- Каждый пользователь в день:
 - 10 просмотров ленты (каждый = 1 HTTP запрос к API)
 - 2 лайка
 - 0.1 поста
 - 5 загрузок изображений профилей

Полезные константы: секунд в сутках $\approx 10^5$, в месяце $\approx 2.5 \times 10^6$, в году $\approx 3 \times 10^7$.

RPS (Requests Per Second) — пользовательские HTTP-запросы:

$$\text{RPS} = \frac{100 \times 10^6 \times 10}{10^5} \approx 10000 \text{ RPS}$$

QPS (Queries Per Second) — запросы к БД:

- Просмотр ленты: 1 запрос на выборку постов + 1 на подсчёт лайков = 2 запроса к БД
- Лайк: 1 INSERT + 1 UPDATE счётчика = 2 запроса к БД

$$\text{Read QPS} = \frac{100 \times 10^6 \times 10 \times 2}{10^5} = 20000 \text{ QPS}$$

$$\text{Write QPS (лайки)} = \frac{100 \times 10^6 \times 2 \times 2}{10^5} = 4000 \text{ QPS}$$

$$\text{Write QPS (посты)} = \frac{100 \times 10^6 \times 0.1}{10^5} = 100 \text{ QPS}$$

TPS (Transactions Per Second) — транзакции с гарантиями ACID:

- Публикация поста требует транзакции (INSERT в posts + UPDATE user stats)

³Internet of Things — сетевое взаимодействие физических устройств с датчиками и контроллерами. Характеризуется высокой частотой передачи небольших порций данных (телеметрия, метрики).

⁴Структурированная информация, описывающая свойства других данных: время создания, размер, тип, владелец, права доступа. Используется для индексирования и маршрутизации без чтения основного содержимого.

⁵Логический контекст взаимодействия клиента с системой между множественными запросами. Хранит состояние пользователя: аутентификацию, корзину, настройки. Реализуется через cookies + серверное хранилище (Redis) или JWT-токены.

⁶Service Level Agreement — соглашение об уровне обслуживания с количественными метриками: uptime (99.9% = 8.76 ч простоя/год), задержка (p99 < 200 мс).

$$\text{TPS} \approx 100 \text{ TPS}$$

IOPS (I/O Operations Per Second) — операции диска:

- Загрузка изображений: запись на диск/object storage

$$\text{IOPS (изображения)} = \frac{100 \times 10^6 \times 5}{10^5} = 5000 \text{ IOPS}$$

Пиковая нагрузка: обычно в 3-5 раз выше средней (вечерние часы, выходные).

2.2.1.5. Оценка объёма хранения

Пример — хранение постов за год:

- 100 QPS записи $\times$ 30M секунд/год = 3 млрд постов
- Средний размер поста: 1 КБ текста + 500 байт метаданных

$$\text{Объём} = 3 \times 10^9 \times 1.5 \text{ КБ} \approx 4.5 \text{ ТБ/год}$$

С учётом индексов и репликации ($\times 3$): 15 ТБ/год.

2.2.1.6. Иерархия памяти

Уровень	Латентность	Пропускная способность
L1 cache	1 нс	1 ТБ/с
L2 cache	4 нс	500 ГБ/с
L3 cache	12 нс	200 ГБ/с
RAM	100 нс	50 ГБ/с
SSD (random)	100 мкс	500 МБ/с
SSD (seq)	50 мкс	3 ГБ/с
HDD (random)	10 мс	1 МБ/с
HDD (seq)	5 мс	200 МБ/с

Случайный доступ на HDD в 200 раз дороже последовательного. На SSD разница меньше (5-10х), но всё ещё значительна.

2.2.1.7. Фундаментальные компромиссы

Read vs Write: оптимизация под чтение замедляет запись и наоборот.

Space vs Time:

- Компрессия** — тратим CPU за уменьшение места на диске
- Денормализация** — тратим место за скорость чтения
- Индексы** — тратим место и замедляем запись за ускорение чтения

2.2.2. Лекция 6: LSM Дерево

2.2.2.1. Общая идея

LSM-дерево решает проблему эффективной записи путём отложенной сортировки и слияния. Вместо немедленной записи на диск в отсортированную структуру, данные буферизуются в памяти и периодически сбрасываются отсортированными порциями.

2.2.2.2. Отсортированная таблица строк (Sorted String Table, SSTable)

SSTable — иммутабельный файл с парами ключ-значение, отсортированными по ключу.

Идея: данные пишутся крупными отсортированными блоками, что делает последовательное чтение быстрым и удобным для диапазонных запросов.

Что решает:

- Дешёвая запись на диск большими порциями
- Быстрые диапазонные чтения благодаря сортировке
- Предсказуемые Ввод-вывод⁷ за счёт последовательного доступа

Что внутри:

- Набор отсортированных блоков данных
- Лёгкий индекс на блоки, чтобы не читать весь файл

Go-псевдоинтерфейс и оценки:

```
// SSTable: иммутабельное чтение по ключу и диапазону.
type SSTable interface {
    // Get: бинарный поиск по индексу + чтение блока.
    //  $O(\log b + k)$ ,  $b$  — число блоков,  $k$  — размер блока.
    Get(key []byte) (value []byte, ok bool)

    // RangeScan: последовательное чтение по диапазону.
    //  $O(\log b + r)$ ,  $r$  — число записей в диапазоне.
    RangeScan(start, end []byte, fn func(k, v []byte))
}
```

2.2.2.3. Memtable

Memtable — изменяемая структура в памяти для накопления записей перед сбросом в SSTable.

Смысл для пользователя: быстрые записи (в памяти), а на диск данные уходят порциями, уже отсортированными.

Что решает:

- Низкая задержка записи
- Сглаживание нагрузки на диск

Что внутри:

- Упорядоченная структура в памяти (обычно Skip List или RB-дерево)
- Быстро поддерживает вставки и поиск по ключу

Go-псевдоинтерфейс и оценки:

```
// Memtable: изменяемые записи, сортировка в памяти.
type Memtable interface {
    // Put: вставка или обновление.
    //  $O(\log m)$ ,  $m$  — число ключей.
    Put(key, value []byte)

    // Get: поиск по ключу.
    //  $O(\log m)$ .
}
```

⁷Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

```

Get(key []byte) (value []byte, ok bool)

// RangeScan: диапазонный проход.
// O(log m + r).
RangeScan(start, end []byte, fn func(k, v []byte))

// Flush: упорядоченный сброс в новый SSTable.
// O(m) на последовательную запись.
Flush(builder SSTableBuilder)
}

```

2.2.2.4. Bloom Filter

Bloom Filter — вероятностная структура для быстрой проверки отсутствия ключа в SSTable.

Зачем он нужен: чтобы не лезть на диск, когда ключа точно нет. Если фильтр говорит “нет”, чтение файлов можно пропустить. Если говорит “возможно”, тогда делаем обычное чтение.

Практический эффект:

- Сильно сокращает Ввод-вывод⁸ при поиске несуществующих ключей
- Используется для каждого SSTable и держится в памяти
- Позволяет экономить дисковые обращения, особенно при большой базе

Что внутри:

- Битовый массив фиксированного размера
- Несколько хеш-функций, выставляющих биты
- Нет ложных отрицаний, но возможны ложные положительные ответы

Go-псевдоинтерфейс и оценки:

```

// BloomFilter: вероятностная проверка наличия.
type BloomFilter interface {
    // Add: устанавливает k битов.
    // O(k), k — число хешей.
    Add(key []byte)

    // MightContain: проверка k битов.
    // O(k).
    MightContain(key []byte) bool
}

```

2.2.2.5. Полная структура LSM-дерева

LSM-дерево — это конвейер для больших объёмов данных, где запись идёт быстро, а порядок поддерживается фоновыми слияниями.

Основные части:

1. WAL — журнал для надёжности: каждый запрос записи сначала фиксируется последовательно.
2. Memtable — быстрая запись и сортировка в памяти.
3. Immutable Memtable — замороженная память, ожидающая сброса.
4. Набор SSTable на диске — отсортированные файлы, каждый с Bloom Filter.

Как работает запись:

- Клиент пишет ключ-значение.
- Запись попадает в WAL и Memtable (быстро, в памяти).
- При переполнении Memtable сбрасывается в новый SSTable (последовательно).

Как работает чтение:

- Поиск идёт от самых свежих структур к старым: Memtable → Immutable → SSTable.

⁸Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

- Перед чтением SSTable проверяется Bloom Filter, чтобы не читать лишнее с диска.
- Результат берётся из первой найденной версии (самой свежей).

Как поддерживается порядок и место:

- На диске постепенно накапливаются SSTable с пересекающимися ключами.
- Фоновая компакция сливает их в более крупные и упорядоченные файлы.
- При слиянии старые версии и записи удаления выкидываются, освобождая место.

Что это даёт пользователю:

- Высокую скорость записи (почти всегда последовательная Ввод-вывод⁹)
- Предсказуемые чтения при больших объёмах данных
- Поддержку диапазонных запросов благодаря сортировке

2.2.2.6. Практическое применение

Ключевое преимущество LSM: очень быстрые записи и высокая пропускная способность на больших объёмах данных за счёт последовательной Ввод-вывод¹⁰ и фоновых слияний.

Хорошо работает для операций:

- Частые вставки и обновления по ключу (write-heavy нагрузки)
- Пакетные записи и стриминг событий
- Диапазонные запросы по ключу (key range scan)
- Чтение свежих данных, когда важна скорость записи и допустима амортизированная задержка

Плохо подходит для:

- Случайных чтений по множеству ключей без кэша (много SSTable)
- Низкой и стабильной задержки чтения в р99 (компакция и пересечения)
- Частых операций удаления, если критично немедленно освободить место

Где в реальной жизни встречается:

- Журналы событий, аналитические события и трекинг поведения
- Метрики, тайм-серии, логи (высокий поток записи)
- Key-Value хранилища (конфиги, сессии, кеш с долговременной персистентностью)
- Большие базы, где критична скорость записи и последовательное чтение

Типичные системы на этой идее: LevelDB/RocksDB, Cassandra, HBase, Bigtable.

⁹Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

¹⁰Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

2.2.3. Лекция 7: В-дерево: индексирование на диске

В-дерево решает проблему эффективного использования кэша диска и минимизации случайных обращений путём группировки данных в узлы фиксированного размера, соответствующие блокам диска.

2.2.3.1. В-фактор и структура узла

В-фактор — это максимальное количество указателей на детей в одном узле (или эквивалентно, максимальное количество ключей + 1).

Внутренняя структура узла порядка B :

- Ключи: $[k_1, k_2, \dots, k_{B-1}]$ в отсортированном порядке (максимум $B - 1$ ключей)
- Указатели на детей: $[p_0, p_1, \dots, p_B]$ где p_i указывает на поддерево с ключами в диапазоне $[k_i, k_{i+1})$
- Листовой узел содержит значения вместо указателей на поддеревья
- Каждый узел занимает ровно одно дисковое обращение — это ключевая идея

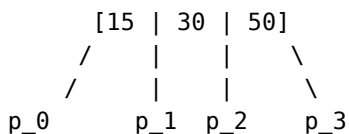
Выбор В-фактора:

- В выбирается так, чтобы узел точно поместился в один блок диска
- Если блок диска = 4 КБ, а каждая пара (ключ, указатель) занимает 10 байт, то $B \approx 400 - 500$
- Маленький B (например, $B=4$): много уровней дерева, много обращений к диску
- Большой B (например, $B=500$): глубина дерева $\mathcal{O}(\log_B n)$ очень мала (для 1 млн записей при $B=500$ глубина всего 3-4 уровня), но поиск внутри узла медленнее

Инвариант В-дерева:

- Все листья находятся на одной глубине h (в отличие от AVL, где глубина может отличаться на 1)
- Каждый внутренний узел содержит от $\lceil \frac{B}{2} \rceil$ до $B - 1$ ключей
- Корень содержит минимум 1 ключ

Пример узла В-дерева при $B=4$:



p_0: ключи < 15
p_1: ключи в [15, 30)
p_2: ключи в [30, 50)
p_3: ключи >= 50

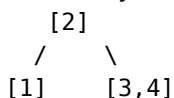
Пример роста В-дерева порядка $B=3$ при вставке 1,2,3,4,5:

Вставляем 1: [1]

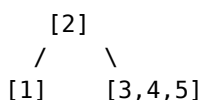
Вставляем 2: [1,2]

Вставляем 3: [1,2,3]

Вставляем 4: узел переполнен! Разделяем:



Вставляем 5:



При вставке нового элемента в переполненный узел:

- Узел разделяется на две половины
- Медиана поднимается в родителя

- Если родитель тоже переполнен, рекурсивно разделяем его
- Это может привести к вырастанию корня и увеличению высоты дерева на 1
- Create – $\mathcal{O}(\log_B n)$
 - Поиск листового узла $\mathcal{O}(\log_B n)$ дисковых обращений
 - Вставка ключа-значения в лист $\mathcal{O}(1)$
 - При переполнении узла: разделение на две половины и поднятие медианы $\mathcal{O}(\log_B n)$ обращений вверх по дереву
- Read – $\mathcal{O}(\log_B n)$
 - Бинарный поиск по ключам текущего узла $\mathcal{O}(\log B)$ — в памяти, быстро
 - Переход к дочернему узлу через одно дисковое обращение
 - Повторение до листа: максимум $\mathcal{O}(\log_B n)$ обращений
- Update – $\mathcal{O}(\log_B n)$
 - Поиск записи $\mathcal{O}(\log_B n)$
 - Обновление значения in-place (B-дерево не версионировано, в отличие от LSM)
- Delete – $\mathcal{O}(\log_B n)$
 - Поиск и удаление ключа $\mathcal{O}(\log_B n)$
 - При недостаточном количестве ключей: слияние с соседним узлом или перемещение ключей из соседа
- Range Query – $\mathcal{O}(\log_B n + \frac{R}{B})$, R — размер результата
 - Поиск левой границы $\mathcal{O}(\log_B n)$
 - Последовательное сканирование листьев (один узел = один читаемый блок диска)
 - Эффективно благодаря локальности: листья часто находятся рядом на диске

Преимущества относительно LSM:

- Нет компакции, стабильная задержка
- Меньше I/O амортизировано: B больше, чем типичный размер блока

2.2.3.2. Практическое применение

Ключевое преимущество B-дерева: предсказуемая и низкая задержка чтения/обновления за счёт прямого доступа к узлам на диске.

Хорошо работает для операций:

- Случайные чтения по ключу с жёсткими требованиями к задержке
- Чтение и обновление отдельных записей in-place
- Смешанные нагрузки (примерно равный read/write)
- Транзакционные OLTP¹¹ базы данных

Плохо подходит для:

- Очень высоких скоростей записи (вставка вызывает дробление узлов)
- Сценариев с большим числом последовательных вставок, где важна чистая пропускная способность
- Стриминга событий, где запись идёт непрерывным потоком

Где в реальной жизни встречается:

- Реляционные БД с индексами: PostgreSQL, MySQL (InnoDB)
- Файловые системы и файловые индексы
- OLTP-системы с частыми точечными запросами и обновлениями

Недостатки:

- Запись требует read-modify-write: чтение узла → изменение → запись обратно
- Сложность балансировки при вставке и удалении

¹¹Online Transaction Processing — класс систем обработки транзакций в реальном времени. Характеризуется большим количеством коротких транзакций (чтение, вставка, обновление, удаление), строковым хранением данных и требованиями к консистентности.

- Для частых записей LSM может затратить сильно меньше Ввод-вывод¹²

¹²Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

2.2.4. Лекция 8: ACID, уровни изоляции, аналитические и транзакционные хранилища

2.2.4.1. Транзакции

Транзакция — Последовательность операций чтения и записи в базе данных, рассматриваемая как единая логическая единица работы. Гарантирует атомарность выполнения: либо все операции завершаются успешно и фиксируются (COMMIT), либо при ошибке все изменения откатываются (ROLLBACK).

```
BEGIN;  
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT; -- или ROLLBACK для отката
```

2.2.4.2. ACID (Atomicity, Consistency, Isolation, Durability)

- **Atomicity** — транзакция выполняется полностью или не выполняется вовсе; реализуется через WAL
- **Consistency** — поддерживаются ограничения целостности (foreign keys, constraints)
- **Isolation** — параллельные транзакции не влияют друг на друга; регулируется уровнями изоляции
- **Durability** — изменения сохраняются навсегда после COMMIT; обеспечивается fsync

2.2.4.3. Транзакционные и аналитические хранилища

OLTP — Online Transaction Processing — класс систем обработки транзакций в реальном времени. Характеризуется большим количеством коротких транзакций (чтение, вставка, обновление, удаление), строковым хранением данных и требованиями к консистентности.

- Паттерн: много мелких операций чтения/записи отдельных записей
- Примеры запросов:
 - SELECT * FROM users WHERE id = 123
 - UPDATE orders SET status = 'shipped' WHERE id = 456
 - INSERT INTO transactions (user_id, amount) VALUES (789, 100.50)
- Строковое хранение (row-oriented): вся строка читается за один блок диска
- СУБД¹³: PostgreSQL, MySQL, Oracle
- Применение: интернет-магазины, банковские системы, CRM¹⁴

OLAP — Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объёмам данных, использует колоночное хранение для оптимизации производительности аналитики.

- Паттерн: большие агрегирующие запросы по миллионам строк
- Примеры запросов:
 - SELECT country, SUM(revenue) FROM sales GROUP BY country
 - SELECT date, COUNT(*) FROM events WHERE timestamp > '2024-01-01' GROUP BY date
 - Анализ трендов, построение отчётов, data science
- Колоночное хранение (column-oriented): колонки хранятся последовательно
 - Читаем только нужные колонки (не всю строку)
 - Лучше сжатие: однотипные данные в одном блоке
 - Векторизованные вычисления на CPU
- СУБД¹⁵: ClickHouse, Apache Druid, Vertica, Snowflake
- Применение: аналитика метрик, BI¹⁶-дашборды, Data Warehouses
- Недостатки: медленные вставки и UPDATE/DELETE отдельных записей

¹³Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

¹⁴Customer Relationship Management — система управления взаимоотношениями с клиентами. Программное обеспечение для автоматизации стратегий взаимодействия с клиентами, включая продажи, маркетинг, техподдержку и аналитику клиентских данных.

¹⁵Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

2.2.4.4. Сжатие колонок

- **Run-Length Encoding** — $[5, 5, 5, 3, 3, 7] \rightarrow [(5, 4), (3, 2), (7, 1)]$
- **Dictionary Encoding** — словарь уникальных значений + индексы
- **Bit Packing** — минимум битов на число
- **Delta Encoding** — разности между значениями для timestamp, id
- **Frame of Reference** — вычитание минимума + bit packing

¹⁶Business Intelligence — технологии и практики сбора, интеграции, анализа и представления бизнес-информации. Включает инструменты для создания отчётов, дашбордов, визуализации данных и поддержки принятия управленческих решений на основе данных.

2.2.5. Лекция 9: Хранилища в оперативной памяти

2.2.5.1. Аналитика: DuckDB и векторизация

Векторизованное выполнение — обработка батчами 1000-10000 строк вместо по одной:

- SIMD инструкции CPU (Single Instruction Multiple Data)
- Лучшее утилизация кэш-линий процессора
- Меньше overhead на интерпретацию

DuckDB — встраиваемая аналитическая СУБД¹⁷:

```
duckdb.sql("SELECT * FROM 'data.parquet' WHERE price > 100")
```

- Чтение Parquet, CSV, JSON без импорта
- Применение: ETL, data science, аналитика

2.2.5.2. Кэширование: Redis

Хранилище в оперативной памяти (in-memory key-value store).

Архитектура:

- Однопоточная модель + event-driven I/O (epoll/kqueue)
- Нет overhead на синхронизацию

Структуры данных:

- **Skip List** (sorted sets) — уже рассмотрен в LSM; `ZADD leaderboard 100 "player1"`
- **Incremental Rehashing** — постепенное перехэширование без блокировки
- **HyperLogLog** — вероятностный подсчёт уникальных; 12 КБ для миллиардов элементов
- Strings, Lists, Sets, Hashes, Bitmaps, Streams

Применение: кэширование, Сессия¹⁸, очереди, real-time аналитика

¹⁷Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

¹⁸Логический контекст взаимодействия клиента с системой между множественными запросами. Хранит состояние пользователя: аутентификацию, корзину, настройки. Реализуется через cookies + серверное хранилище (Redis) или JWT-токены.

2.3. Модуль 3: Методы поиска похожих/близких элементов

2.3.1. Лекция 10: Специализированные индексы для поиска

2.3.1.1. Inverted Index (перевёрнутые индексы)

В прошлом модуле мы изучили прямой индекс, который ищет данные по ключу (см. В-дерево). Основная идея обратного индекса: хранить для каждого термина список документов, где он встречается. Это позволяет быстро находить документы по словам запроса.

Go-псевдоинтерфейс и оценки:

```
// InvertedIndex: термин -> список документов.
type InvertedIndex interface {
    // Add: добавить документ в индекс.
    // O(t), t — число терминов в документе.
    Add(docID int, terms []string)

    // QueryAND: пересечение posting lists.
    // O(sum L_i), L_i — длины списков терминов.
    QueryAND(terms []string) []int

    // QueryOR: объединение posting lists.
    // O(sum L_i).
    QueryOR(terms []string) []int
}
```

Внутреннее устройство:

- Словарь терминов: term -> смещение posting list на диске
- Posting list: отсортированный список docID с частотами и позициями

Пример индекса (термины выделены цветом):

```
cat -> [1, 3]
sat -> [1, 2]
dog -> [2]
ran -> [3]
```

Добавление документа Doc4: “cat sat sat”:

```
cat -> [1, 3, 4]
sat -> [1, 2, 4]    // tf=2, positions=[2,3] для Doc4
```

Удаление Doc2 (логическое):

```
dog -> [2]    // docID=2 помечен удалённым
sat -> [1, 2] // docID=2 остается до пересборки/слияния
```

Фразовый запрос “cat sat”:

- 1) пересечение списков cat и sat -> [1]
- 2) проверка позиций: cat@1, sat@2 -> фраза совпала

Практическое применение:

- Полнотекстовый поиск: слова, фразы
- Быстрые запросы по большим корпусам документов
- Поиск по логам/документации/каталогам
- Системы: Elasticsearch, Lucene, Solr

2.3.1.2. Trie (префиксные деревья)

Основная идея: хранить ключи посимвольно, чтобы быстро отвечать на запросы по префиксу.

Go-псевдоинтерфейс и оценки:

```
// Trie: поиск по префиксу.
type Trie interface {
```



```

// Insert: добавить слово.
// O(L), L — длина слова.
Insert(word string)

// Contains: точный поиск.
// O(L).
Contains(word string) bool

// StartsWith: получить все слова по префиксу.
// O(P + r), P — длина префикса, r — число результатов.
StartsWith(prefix string) []string
}

```

Внутреннее устройство:

- Узлы по символам, переходы по алфавиту
- Маркер окончания слова
- Компрессия (radix trie) снижает память

Пример построения:

Вставляем "car":

```
root -> c -> a -> r (*)
```

Вставляем "cat":

```
root -> c -> a -> r (*)
      \-> t (*)
```

Поиск префикса "ca":

- 1) идём root -> c -> a
- 2) обходим поддерево и получаем: car, cat

Удаление "car":

- 1) снимаем маркер окончания с r
- 2) если у r нет детей, удаляем r и поднимаемся вверх

Практическое применение:

- Автодополнение, подсказки, поиск по префиксу
- Поиск по словарям, маршрутизация по строковым ключам

2.3.1.3. Bitmap indexes

Основная идея: для каждого значения категории хранить битовый массив, где 1 означает, что запись содержит это значение.

Go-псевдоинтерфейс и оценки:

```

// BitmapIndex: значение -> битовый вектор.
type BitmapIndex interface {
    // Set: установить бит для записи.
    // O(1).
    Set(value string, rowID int)

    // And/Or: булевы операции над выборками.
    // O(n/word), n — число строк.
    And(values []string) []int
    Or(values []string) []int
}

```

Внутреннее устройство:

- Для каждого значения хранится битсет длиной = числу строк
- Быстрые битовые операции AND/OR/NOT по словам машинного слова
- Сжатие битсетов (Roaring, WAH) для экономии памяти

Практическое применение:

- Фильтры по категориальным полям (status, country, type)
- Аналитические запросы и агрегации
- Колоночные хранилища и OLAP¹⁹ системы

2.3.2. Лекция 11: Векторные индексы для поиска подобия

2.3.2.1. Основы векторных представлений

Идея: объект (текст, изображение, товар) переводится в вектор чисел так, чтобы близкие по смыслу объекты были близки по расстоянию.

Go-псевдоинтерфейс и оценки:

```
// VectorStore: хранение и поиск по близости.
type VectorStore interface {
    // Add: добавить вектор в индекс.
    // O(d), d — размерность.
    Add(id int, vec []float32)

    // Search: найти k ближайших и вернуть id + метрику близости.
    // O(k * log n) амортизировано для индекса, O(n*d) для полного скана.
    Search(query []float32, k int) []Neighbor
}

type Neighbor struct {
    ID      int
    Score   float32 // расстояние или similarity
}
```

Внутренняя идея:

- Векторы фиксированной размерности (например, 384/768)
- Индекс нужен, чтобы не сравнивать запрос со всеми векторами

Метрики близости:

Косинусное сходство (Cosine similarity)

$$\cos(q, v) = \frac{q \cdot v}{\|q\| \cdot \|v\|}$$

- Смысл: сравнивает направление, не зависит от длины вектора
- Когда использовать: семантический поиск, где длина текста не должна влиять на близость
- Пример: запрос “как вернуть товар” должен найти и короткую карточку FAQ, и длинную статью поддержки про возвраты — обе про один смысл, хотя объем текста разный
- Пример: поиск похожих запросов в Support²⁰-тикетах: короткие и длинные формулировки про один дефект должны совпадать

Евклидово расстояние L2 (L2 distance)

$$\|q - v\|_2 = \sqrt{\sum_i (q_i - v_i)^2}$$

- Смысл: абсолютное расстояние между точками в пространстве
- Когда использовать: когда масштаб признаков имеет значение и “дальше” должно быть строго хуже
- Пример: поиск похожих товаров по числовым характеристикам (вес, размеры, объем батареи). Большие отличия по одному из параметров должны увеличивать расстояние
- Пример: поиск похожих изображений по Эмбединг²¹ из модели, где длина вектора несет информацию о интенсивности/яркости и ее нельзя игнорировать

¹⁹Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объемам данных, использует колоночное хранение для оптимизации производительности аналитики.

²⁰Служба поддержки и обработка обращений пользователей: тикеты, инциденты, запросы на возврат и консультации.

Скалярное произведение (Dot product)

$$q \cdot v = \sum_i q_i v_i$$

- Смысл: близость растет, если векторы “смотрят” в одну сторону и имеют большую длину
- Когда использовать: когда длина вектора отражает силу сигнала (уверенность, популярность)
- Пример: рекомендательная система — вектор пользователя и вектор товара. Большой модуль товара (популярный) усиливает сходство
- Пример: при ранжировании контента важен не только смысл, но и вес/сила признака (например, частота взаимодействий)

Пример:

```
q = [1.0, 0.0]
a = [0.9, 0.1] // близко к q
b = [-0.2, 0.9] // далеко от q
```

$\cos(q, a) \sim 0.99$, $\cos(q, b) \sim -0.2$

Практическое применение:

- Семантический поиск по текстам и документам
- Поиск похожих товаров/изображений/песен
- Рекомендации и кластеризация

2.3.2.2. HNSW (Hierarchical Navigable Small World)

Идея: построить многослойный граф, где верхние уровни редкие и помогают быстро найти область, а нижние дают точное окружение.

Go-псевдоинтерфейс и оценки:

```
// HNSW: графовый индекс ближайших соседей.
type HNSW interface {
    // Add: вставка с поиском входной точки.
    // O(M * log n) амортизировано.
    Add(id int, vec []float32)

    // Search: greedy поиск по слоям.
    // O(M * log n) амортизировано.
    Search(query []float32, k int) []int
}
```

Внутреннее устройство:

- Несколько уровней графа (L0, L1, L2...), сверху мало узлов
- Каждый узел хранит список соседей (degree M)
- Поиск: на верхнем уровне делаем грубый поиск, затем спускаемся ниже

Пример:

```
L2:  [A]----[B]
      \    |
L1:  [C]--[D]--[E]
      \   \  \
L0:  [F]--[G]--[H]--[I]
```

Запрос начинается на L2, быстро находит область, затем уточняет на L1/L0.

Практическое применение:

- Семантический поиск с большими коллекциями (миллионы векторов)
- Поиск ближайших соседей в рекомендациях
- Быстрый ANN-поиск с управляемой точностью

²¹Векторное представление объекта (текста, изображения, пользователя, товара), полученное моделью машинного обучения для измерения семантической близости через расстояния в векторном пространстве.

2.3.2.3. LSH (Locality-Sensitive Hashing)

Идея: хешировать вектора так, чтобы похожие объекты с высокой вероятностью попадали в один бакет.

Go-псевдоинтерфейс и оценки:

```
// LSH: вероятностный поиск ближайших.
type LSH interface {
    // Add: добавить вектор в бакеты.
    //  $O(L * k * d)$ ,  $L$  — число таблиц,  $k$  — число хешей.
    Add(id int, vec []float32)

    // Search: искать в бакетах запроса.
    //  $O(L * k * d + c * d)$ ,  $c$  — кандидаты.
    Search(query []float32, k int) []int
}
```

Внутреннее устройство:

- Несколько хеш-таблиц, каждая с набором случайных гиперплоскостей
- Вектор попадает в бакет по знаку скалярных произведений
- Запрос собирает кандидатов из бакетов и точно пересчитывает расстояния

Пример (2D, одна гиперплоскость):

Плоскость: $x - y = 0$
 $v1 = (2, 1) \rightarrow \text{hash } 1$, $v2 = (1, 2) \rightarrow \text{hash } 0$
Похожие векторы чаще попадают в один hash.

Практическое применение:

- Быстрый грубый поиск по огромным коллекциям
- Сценарии, где допустимы вероятностные ответы
- Предфильтрация кандидатов перед точным ранжированием

2.3.2.4. Примеры: Weaviate, Pinecone, Qdrant

Практические системы:

- Weaviate: open-source векторная БД, GraphQL/REST, HNSW по умолчанию
- Pinecone: managed сервис, автошардирование и масштабирование
- Qdrant: open-source, HNSW + фильтры, удобен для self-hosted

Где встречается в реальных продуктах:

- Поиск по базе знаний в чат-ботах
- Поиск похожих товаров и контента
- Поиск по логам и тикетам поддержки

2.3.3. Лекция 12: Геопространственные структуры

2.3.3.1. R-Tree

Идея: группировать объекты по минимальным ограничивающим прямоугольникам (MBR), чтобы быстро отсекал области.

Go-псевдоинтерфейс и оценки:

```
// RTree: индекс для прямоугольников и точек.
type RTree interface {
    // Insert: вставка объекта с его MBR.
    //  $O(\log n)$  амортизировано.
    Insert(id int, rect Rect)

    // Search: поиск пересечений с областью.
    //  $O(\log n + r)$ ,  $r$  — число найденных объектов.
    Search(area Rect) []int
}
```

Внутреннее устройство:

- Узлы содержат несколько MBR и указатели на детей
- Каждый узел соответствует прямоугольнику, покрывающему все дочерние объекты
- Поиск: обход только тех узлов, чьи MBR пересекаются с запросом

Пример:

R1: [0,0] - [5,5] содержит A, B

R2: [6,0] - [9,4] содержит C

Запрос: [4,1] - [7,3] пересекает R1 и R2 -> проверяем A, B, C

Практическое применение:

- Геопоиск объектов в радиусе и по прямоугольнику
- Карты, геосервисы, логистика
- Системы: PostGIS, SQLite R-Tree

2.3.3.2. QuadTree и KD-Tree

Идея: разбиение пространства для ускорения поиска по координатам.

QuadTree:

- Делит плоскость на 4 квадранта до тех пор, пока в ячейке мало объектов

KD-Tree:

- Делит пространство гиперплоскостями, поочередно по осям (x, y, z...)

Go-псевдоинтерфейс и оценки:

```
// SpatialIndex: общий интерфейс.  
type SpatialIndex interface {  
    Insert(id int, p Point)           // O(log n) амортизировано  
    RangeSearch(area Rect) []int      // O(log n + r)  
    Nearest(p Point, k int) []int     // O(log n + k) в среднем  
}
```

Пример QuadTree:

Плоскость -> 4 квадранта

Точка (1,1) идет в Q1, (8,1) в Q2, (2,7) в Q3...

Запрос по области проверяет только затронутые квадранты.

Пример KD-Tree:

Разбиение:

уровень 1: по x

уровень 2: по y

уровень 3: по x ...

Поиск ближайших идет по веткам с возможным откатом.

Практическое применение:

- Геопоиск, карты, игры, физика и коллизии
- Индексация точек (POI), поиск ближайших объектов
- KD-Tree хорош для ближайших соседей, QuadTree — для равномерной сетки на карте

2.4. Модуль 4: Операционные компоненты

2.4.1. Лекция 13: Прокси и Rate Limiting

2.4.1.1. Forward Proxy / Reverse Proxy

2.4.1.2. Распределение нагрузки

- Round Robin
- Least Connections
- Weighted Least Connections

- IP Hash
- Consistent Hashing

2.4.1.3. Ограничение частоты запросов

- Token Bucket
- Sliding Window

2.4.2. Лекция 14: Безопасность и управление доступом

2.4.2.1. Аутентификация и авторизация

2.4.2.2. RBAC и ABAC

2.4.2.3. Single Sign-On

2.4.3. Лекция 15: Логирование, мониторинг и планирование задач

2.4.3.1. Сбор логов

2.4.3.2. Сбор метрик (Prometheus)

2.4.3.3. Мониторинг и диагностика (Grafana)

2.4.3.4. Планировщики (Celery, Airflow, Dagster)

2.5. Модуль 5: System Design

2.5.1. Лекция 16: Разбор дизайна конкретной системы

2.5.1.1. Применение всех концепций на практике

3. Распределённые системы обработки данных

3.1. Модуль 1: Компоненты распределённых систем

3.1.1. Лекция 1: Основы распределённых систем

3.1.1.1. CAP-теорема

- Следствия компромиссов между согласованностью, доступностью и устойчивостью к разделению сети
- Примеры CA, AP, CP систем

3.1.1.2. Механизмы репликации данных

- Архитектура “ведущий-подчинённый” (Leader-Follower)
- Репликация с использованием кворумов (quorum-based replication)

3.1.1.3. ZooKeeper: распределённое решение проблем согласованности и отказоустойчивости

- Механизмы решения классических проблем распределённых систем
- Применение в системах Kafka и ClickHouse

3.1.2. Лекция 2: Гарантии консистентности и координация распределённых транзакций

3.1.2.1. Алгоритмы достижения консенсуса

- Raft
- Paxos

3.1.2.2. Модель конечной консистентности (Eventual consistency) и требование идемпотентности операций

3.1.2.3. Протокол двухфазного коммита (Two-Phase Commit, 2PC)

- Этапы выполнения
- Гарантии надёжности

3.2. Модуль 2: Распределённые сервисы

3.2.1. Лекция 3: Асинхронное взаимодействие сервисов

3.2.1.1. Event Bus и Event-Driven Architecture

3.2.1.2. Publish-Subscribe vs Message Queue

3.2.1.3. Saga Pattern

3.2.2. Лекция 4: Kafka

3.2.2.1. Внутреннее устройство

3.2.3. Лекция 5: Kubernetes

3.2.3.1. Роль и место в современных архитектурах

3.2.3.2. Области использования

3.2.3.3. Внутреннее устройство

3.2.4. Лекция 6: ClickHouse

3.2.4.1. Архитектура системы хранения и обработки данных

- Принципы организации хранения
- Методы компрессии данных

3.2.4.2. Партиционирование

- Выбор ключа
- Управление партициями (добавление, удаление)

3.2.4.3. Таблицы серии MergeTree как основной механизм персистентности

- Базовая таблица MergeTree и её специализированные варианты
- ReplacingMergeTree для управления версионированием
- ReplicatedMergeTree для обеспечения отказоустойчивости

3.2.5. Лекция 7: Apache Spark

3.2.5.1. Роль и место в современных архитектурах

3.2.5.2. Области использования

3.2.5.3. Внутреннее устройство

3.2.5.4. API

- Dataframe
- SparkSQL

3.2.6. Лекция 8: Erlang

3.2.6.1. Роль и место в современных архитектурах

3.2.6.2. Erlang VM (BEAM) и процессы

3.2.6.3. Модель акторов

3.2.6.4. Fault tolerance и “Let it crash” философия

3.2.6.5. OTP фреймворк (Supervisor, GenServer)

3.2.6.6. Синхронизация и распределённые вычисления

3.2.6.7. Elixir

3.3. Модуль 3: Распределенные БД

3.3.1. Лекция 9: Большие данные

3.3.1.1. Свойства

- Объем
- Скорость
- Разнообразие
- Достоверность

3.3.1.2. Архитектуры

- Modern Data Architecture
- Lambda
- Lakehouse

3.3.2. Лекция 10: Full Text Search в распределённых системах

3.3.2.1. Elasticsearch и Solr

- Sharding: hash-based, range-based
- Replication и replica factor
- Eventual consistency при индексировании

3.3.2.2. Распределённый поиск

- Broadcast query, merge результатов
- Search After вместо offset/limit

3.3.2.3. Масштабирование

- Hot shards проблема

- Index rollover, segment merging

3.3.3. Лекция 11: Документные хранилища

3.3.3.1. MongoDB: sharding strategies, write concerns

3.3.3.2. Object storage: MinIO

3.3.4. Лекция 12: Распределенные пространственные и графовые БД

3.3.4.1. Графовые БД

3.3.4.2. Пространственные БД

3.3.4.3. Шардирование графов и пространственных данных